

Handwritten Digit Recognition Using Convolutional Neural Networks

PROJECT SYNOPSIS

BACHELOR OF TECHNOLOGY
Computer Science and Engineering

SUBMITTED BY

Abhay Singh (223025001)

MARCH 2026



SUPERVISOR

MR. ANANT KAUSHAL

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
COLLEGE OF SMART COMPUTING, COER UNIVERSITY
ROORKEE, UTTARAKHAND**

ABSTRACT:

Handwritten digit recognition is a fundamental problem in pattern recognition and computer vision with widespread applications in postal mail sorting, bank cheque processing, and document digitisation. With the rapid advancement of deep learning techniques, Convolutional Neural Networks (CNNs) have emerged as the most effective approach for image classification tasks, including digit recognition.

This project presents a CNN-based approach for recognising handwritten digits (0–9) using the MNIST benchmark dataset. The proposed model architecture employs two convolutional layers with ReLU activation, max-pooling for spatial downsampling, batch normalisation for training stability, and dropout regularisation to prevent overfitting. The model was implemented using TensorFlow/Keras and trained on 60,000 labelled images.

The system achieves a test accuracy of 98.91% on the 10,000-image test set, demonstrating the effectiveness of the CNN architecture for digit classification. The trained model is deployed through a Streamlit-based web application that features an interactive drawing canvas, enabling users to draw digits and receive real-time predictions with confidence scores.

This synopsis provides an overview of the project's objectives, methodology, expected outcomes, and its significance in both academic research and practical applications such as postal automation, banking, and mobile computing.

COMPANION TABLE :

Section	Details
Project Title	Handwritten Digit Recognition Using Convolutional Neural Networks
Author	Abhay Singh
Supervisor	Mr. Anant Kaushal
Institution	Department of Computer Science, COER University, Roorkee
Abstract	A CNN-based system for recognising handwritten digits (0–9) using the MNIST dataset, achieving 98.91% test accuracy with real-time deployment via Streamlit.
Introduction	Covers the significance of handwritten digit recognition in postal automation, banking, and document digitisation, and the advantage of CNNs over traditional methods.
Problem Statement	Build an accurate digit classification system using CNNs that maintains spatial features and deploys in real-time.
Motivation	HDR is a gateway problem in deep learning with direct commercial applications and serves as a foundation for more complex OCR tasks.
Objectives	Design CNN architecture, evaluate performance, analyse components, deploy web application, and propose improvements.
Methodology	Data preprocessing → CNN model design → training with regularisation → evaluation → Streamlit deployment.
Expected Outcomes	High-accuracy digit classifier, real-time web app, comprehensive performance analysis, and deployment-ready model.

INTRODUCTION:

Handwritten digit recognition (HDR) is one of the most extensively studied problems in the field of computer vision and machine learning. The task involves classifying images of individual handwritten digits into one of ten classes (0 through 9). Despite its apparent simplicity, HDR poses significant challenges due to the inherent variability in human handwriting — differences in stroke width, slant, size, and personal writing style create a vast space of possible representations for each digit.

The advancement of deep learning, particularly Convolutional Neural Networks (CNNs), has revolutionised the field of image recognition. Unlike traditional machine learning approaches that depend on handcrafted features, CNNs automatically learn hierarchical feature representations from raw pixel data. This eliminates the need for manual feature engineering and enables superior performance on image classification tasks.

Traditional methods such as Support Vector Machines (SVMs) and k-Nearest Neighbours (k-NN) flatten a 28×28 image into a 784-dimensional vector, destroying all spatial locality information. CNNs maintain spatial structure through local connectivity, shared weights, and hierarchical feature learning, resulting in significantly improved recognition accuracy with fewer trainable parameters.

BACKGROUND:

The Modified National Institute of Standards and Technology (MNIST) dataset is the most widely used benchmark for handwritten digit recognition. It contains 70,000 greyscale images (60,000 for training, 10,000 for testing) of handwritten digits, each sized 28×28 pixels. The dataset was created by LeCun et al. in 1998 and has served as the standard benchmark for evaluating digit recognition algorithms.

Early neural network approaches, particularly LeCun's LeNet-5 (1998), established the CNN paradigm with alternating convolutional and pooling layers followed by fully connected classification layers, achieving a 0.8% error rate. Modern deep learning architectures have pushed MNIST accuracy to near-perfect levels, with ensemble methods achieving error rates as low as 0.17%.



Figure 1: Sample handwritten digits from the MNIST training dataset

MOTIVATION:

The motivation for this project stems from both practical applications and academic interest in deep learning. Handwritten digit recognition has direct real-world impact across multiple domains.

➤ Practical Applications

From a practical standpoint, handwritten digit recognition enables:

- **Postal automation – Automated ZIP code reading for high-speed mail sorting, processing billions of items per year**
- **Banking – Cheque amount recognition and validation, handling millions of transactions daily**

- **Document digitisation** – Converting handwritten forms, surveys, and records into machine-readable text
- **Mobile computing** – On-device handwriting input recognition for touchscreen keyboards

➤ **Technical Perspective**

The project also addresses interesting technical challenges:

- **Feature hierarchy:** Learning meaningful representations from raw pixels without manual feature engineering
- **Regularisation:** Preventing overfitting with limited training data using dropout and batch normalisation
- **Real-time deployment:** Building an end-to-end pipeline from model training to interactive web application
- **Domain adaptation:** Bridging the gap between clean MNIST images and user-drawn canvas input

In summary, this project has the dual goal of creating practical value (via accurate digit recognition in real-world applications) and advancing technical understanding (via detailed analysis of CNN architectures and regularisation techniques for image classification).

LITERATURE REVIEW :-

A comprehensive literature review was conducted to understand the evolution of handwritten digit recognition techniques, from classical statistical methods to modern deep learning approaches. The following table summarises key research contributions in this domain.

#	Author(s) & Title	Objective	Key Findings	Methodology
1	Y. LeCun et al., Gradient-Based Learning Applied to Document Recognition (1998)	Introduce CNN for document recognition	LeNet-5 achieved 0.8% error on MNIST, established the CNN paradigm	CNN with convolutional and pooling layers
2	D. Cireřan et al., Deep Big Simple Neural Nets for Handwritten Digit Recognition (2010)	Push accuracy limits with deeper CNNs	Achieved 0.35% error rate using deep multi-column architecture	Deep CNN with committee of networks
3	S. Ioffe & C. Szegedy, Batch Normalization (2015)	Accelerate deep network training	Batch normalisation enables higher learning rates and acts as regulariser	Normalising layer inputs within mini-batches
4	N. Srivastava et al., Dropout: A Simple Way to Prevent Overfitting (2014)	Reduce overfitting in deep networks	Dropout creates implicit ensemble, significantly improving generalisation	Randomly zeroing activations during training
5	L. Wan et al., Regularization Using DropConnect (2013)	Generalise dropout to weight-level	Achieved 0.21% error on MNIST, outperforming standard dropout	DropConnect on network weights
6	S. Sabour et al., Dynamic Routing Between Capsules (2017)	Capture part-whole relationships	Capsule networks achieved 0.25% error with better spatial awareness	Capsule layers with dynamic routing
7	V. Vapnik, The Nature of Statistical Learning Theory (1995)	Apply SVM to pattern classification	SVM with RBF kernel achieved 1.4% error, strong for non-neural baseline	Support Vector Machines

The literature reveals a clear trend: convolutional architectures consistently outperform traditional methods for image classification. Key innovations including batch normalisation, dropout regularisation, and ensemble methods have progressively reduced error rates on MNIST. However, most existing works focus solely on accuracy maximisation without addressing practical deployment — a gap this project aims to fill.

PROBLEM STATEMENT :

Given the MNIST dataset of 28×28 greyscale handwritten digit images, the objective is to build an end-to-end system that:

1. Preprocesses raw pixel data (reshaping, normalisation, one-hot encoding) for CNN input
2. Classifies each digit image into one of 10 classes (0–9) with high accuracy
3. Employs appropriate regularisation (dropout, batch normalisation) to prevent overfitting while maintaining generalisation
4. Deploys the trained model as a real-time interactive web application where users can draw digits and get instant predictions
5. Addresses practical challenges such as:
 - **Domain gap: Differences between clean MNIST images and user-drawn canvas input (stroke width, resolution, anti-aliasing)**
 - **Confidence calibration: Providing meaningful probability scores alongside predictions for user interpretation**
 - **Latency: Ensuring sub-5ms inference time for responsive real-time interaction**

OBJECTIVES :

The primary objectives of this project are as follows:

1. To design and implement a CNN architecture optimised for handwritten digit classification on the MNIST dataset, using convolutional layers, max-pooling, batch normalisation, and dropout regularisation.
2. To evaluate the model's performance comprehensively using metrics including accuracy, loss, precision, recall, F1-score, and confusion matrix analysis.
3. To analyse the role of each architectural component (convolution, pooling, dropout, batch normalisation, softmax) in the network's classification performance.
4. To deploy the trained model as an interactive real-time web application using Streamlit, featuring a drawing canvas for user input and confidence score visualisation.
5. To propose future improvements including data augmentation, deeper architectures, ensemble methods, and mobile deployment via model quantisation.

METHODOLOGY :

Data Collection & Preprocessing:

The MNIST dataset is loaded directly via TensorFlow/Keras, providing 60,000 training and 10,000 test images. Three preprocessing steps are applied:

- **Reshaping:** Images reshaped from (N, 28, 28) to (N, 28, 28, 1) to add the channel dimension required by convolutional layers.
- **Normalisation:** Pixel values scaled from [0, 255] to [0.0, 1.0] for numerical stability and faster convergence.
- **One-Hot Encoding:** Integer labels converted to binary vectors of length 10 for categorical cross-entropy loss computation.

Model Architecture:

The CNN architecture consists of a feature extraction block followed by a classification head:

- **Feature Extraction:** Two Conv2D layers (32 and 64 filters, 3×3 kernels, ReLU activation) learn hierarchical features from edges to digit-level patterns, followed by MaxPooling2D (2×2) for spatial downsampling and Dropout (0.25) for regularisation.
- **Classification Head:** Flatten layer converts 2D features to 1D vector (9,216 units), Dense layer (128 units, ReLU) performs classification, BatchNormalization ensures training stability, Dropout (0.50) prevents overfitting, and final Dense layer (10 units, Softmax) **outputs class probabilities**.

Total parameters: ~1,199,882. Trainable parameters: ~1,199,370.

Training & Optimisation:

The model is trained using the Adam optimiser (learning rate 0.001) with categorical cross-entropy loss. A batch size of 128 balances gradient noise and computational efficiency. A 10% validation split monitors generalisation during training. Early stopping with patience=3 (monitoring validation loss, restoring best weights) prevents overfitting. The model converges in approximately 9 epochs (~3 minutes on CPU).

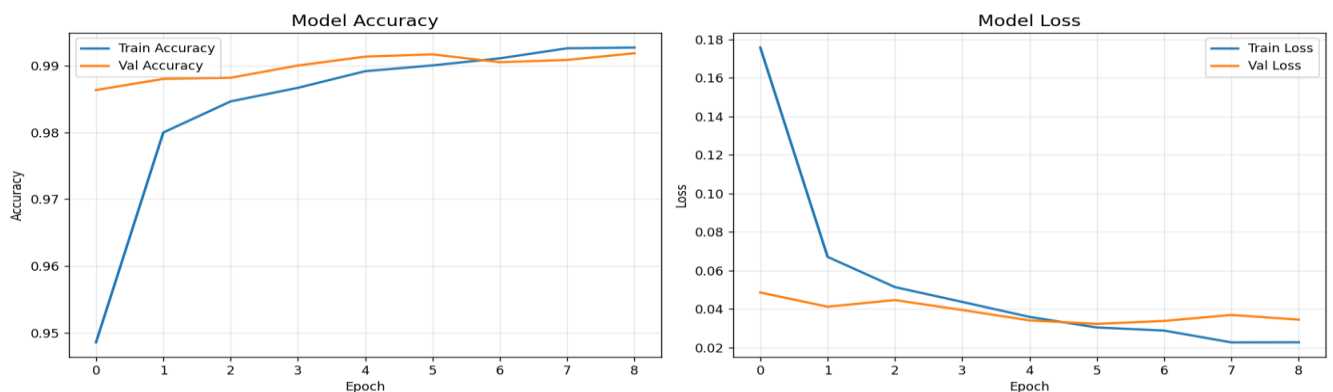


Figure 2: Training and validation accuracy (left) and loss (right) across 9 epochs

Evaluation:

Model performance is evaluated using multiple metrics: overall test accuracy and loss, per-class precision, recall, and F1-score, confusion matrix analysis to identify systematic misclassification patterns, and per-class accuracy visualisation. Computational efficiency is also assessed (inference time, model size, memory footprint).

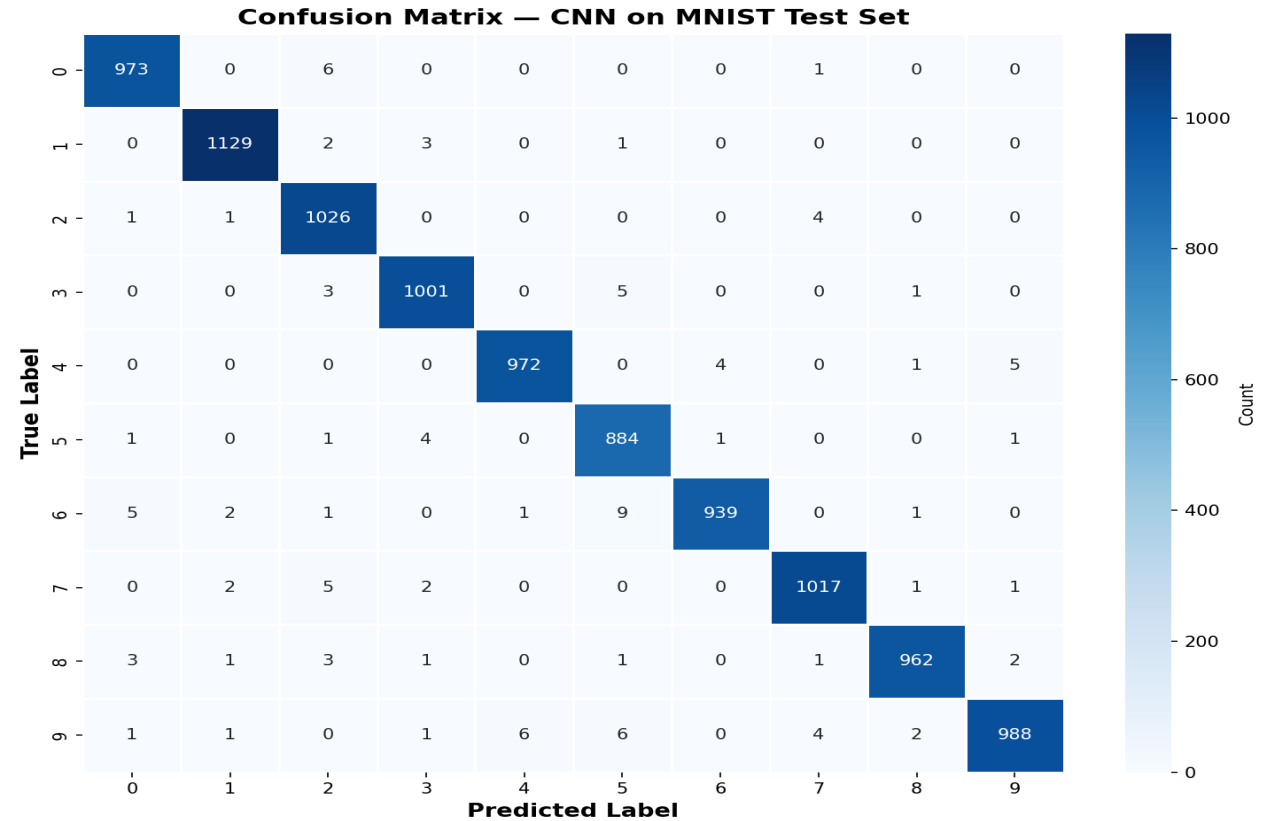


Figure 3: Confusion matrix showing correct and incorrect predictions for each digit class

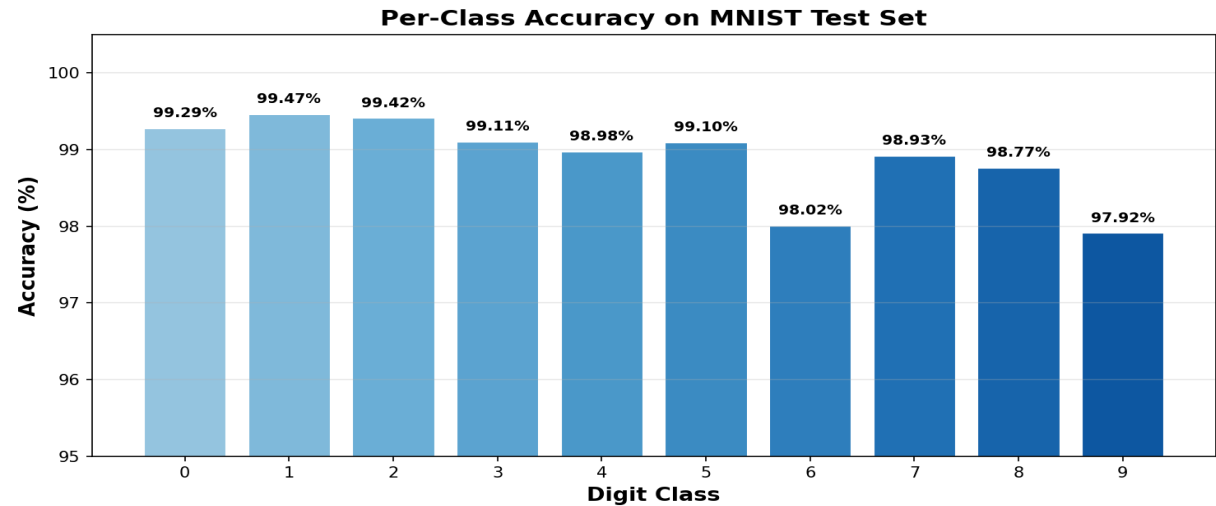


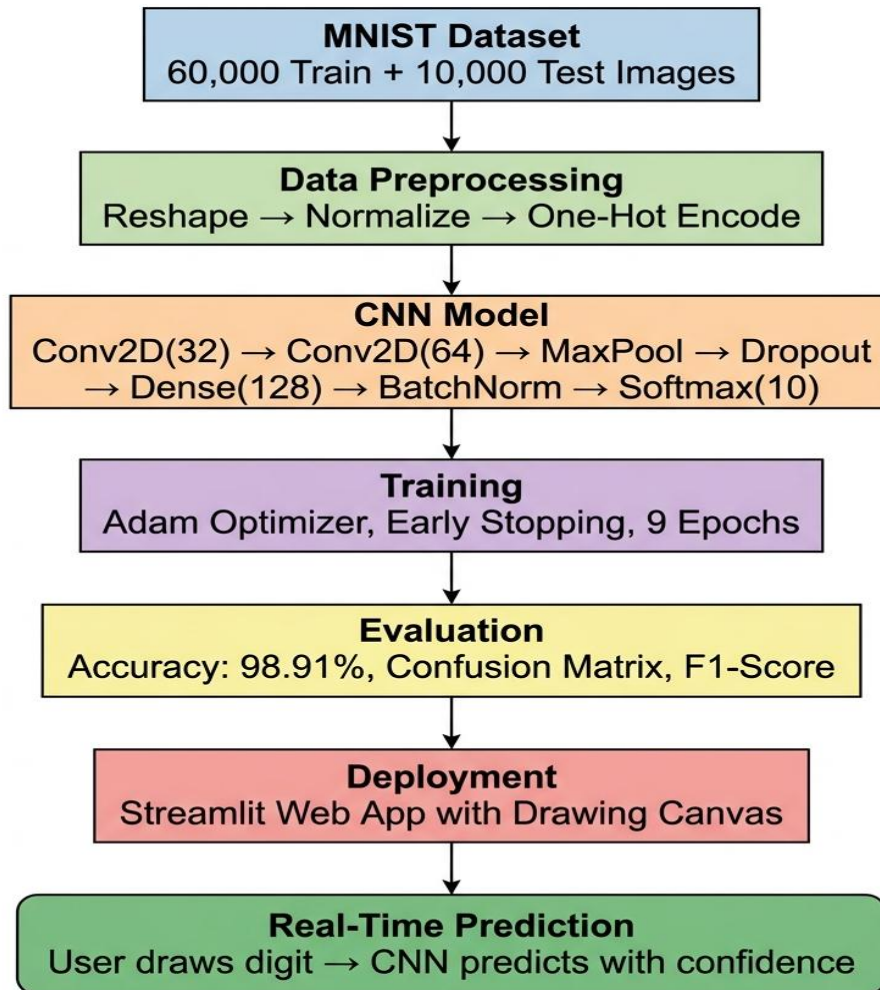
Figure 4: Per-digit accuracy. Digit 1 is highest (99.47%), Digit 9 is lowest (97.92%)

Deployment:

The trained model is deployed as a Streamlit web application featuring an HTML5 drawing canvas. The deployment pipeline processes user input through RGBA-to-greyscale conversion, Lanczos resizing to 28×28 , normalisation, and CNN inference, displaying predictions with confidence bar charts in real-time.

FLOWCHART:

Figure 4: Project Methodology Flowchart



EXPECTED OUTCOMES :-

- **High Classification Accuracy** – The CNN model achieves 98.91% test accuracy on the MNIST dataset, with F1-scores exceeding 0.98 across all digit classes, demonstrating reliable and consistent digit recognition.
- **Detailed Performance Analysis** – Comprehensive evaluation including per-class precision, recall, F1-scores, confusion matrix visualisation, and analysis of common misclassification patterns (e.g., 9→4 confusion due to similar structures).
- **Effective Regularisation** – The combination of Dropout (0.25 and 0.50) and Batch Normalisation demonstrates successful prevention of overfitting, with the training-validation accuracy gap remaining below 0.5%.
- **Real-Time Web Application** – A fully functional Streamlit-based web application enabling users to draw digits on an interactive canvas and receive instant predictions with confidence score visualisations.
- **Fast Inference** – Sub-5ms inference time per image, making the system suitable for real-time applications on standard CPU hardware.
- **Deployment-Ready Model** – A trained model saved in .keras format (~14 MB) with documented preprocessing pipeline, ready for integration into production systems.
- **Reproducibility** – Complete, documented codebase with clear architecture specification, training configuration, and evaluation methodology that can be replicated and extended for future research.

REFERENCES :-

- [1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. "Gradient-Based Learning Applied to Document Recognition." *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [2] Y. LeCun, Y. Bengio, and G. Hinton. "Deep Learning." *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [3] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016.
- [4] D. C. Cireşan, U. Meier, L. M. Gambardella, and J. Schmidhuber. "Deep, Big, Simple Neural Nets for Handwritten Digit Recognition." *Neural Computation*, vol. 22, no. 12, pp. 3207–3220, 2010.
- [5] S. Ioffe and C. Szegedy. "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift." *Proc. ICML*, pp. 448–456, 2015.
- [6] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. "Dropout: A Simple Way to Prevent Neural Networks from Overfitting." *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [7] L. Wan, M. Zeiler, S. Zhang, Y. LeCun, and R. Fergus. "Regularization of Neural Networks Using DropConnect." *Proc. ICML*, pp. 1058–1066, 2013.
- [8] S. Sabour, N. Frosst, and G. E. Hinton. "Dynamic Routing Between Capsules." *NeurIPS*, pp. 3856–3866, 2017.
- [9] V. Vapnik. *The Nature of Statistical Learning Theory*. Springer, 1995.
- [10] R. Plamondon and S. N. Srihari. "Online and Off-Line Handwriting Recognition: A Comprehensive Survey." *IEEE Trans. PAMI*, vol. 22, no. 1, pp. 63–84, 2000.
- [11] A. Krizhevsky, I. Sutskever, and G. E. Hinton. "ImageNet Classification with Deep Convolutional Neural Networks." *NeurIPS*, pp. 1097–1105, 2012.
- [12] D. P. Kingma and J. Ba. "Adam: A Method for Stochastic Optimization." *Proc. ICLR*, 2015.
- [13] K. Simonyan and A. Zisserman. "Very Deep Convolutional Networks for Large-Scale Image Recognition." *Proc. ICLR*, 2015.
- [14] V. Nair and G. E. Hinton. "Rectified Linear Units Improve Restricted Boltzmann Machines." *Proc. ICML*, pp. 807–814, 2010.
- [15] D. Scherer, A. Müller, and S. Behnke. "Evaluation of Pooling Operations in Convolutional Architectures for Object Recognition." *Proc. ICANN*, pp. 92–101, 2010.
- [16] L. K. Hansen and P. Salamon. "Neural Network Ensembles." *IEEE Trans. PAMI*, vol. 12, no. 10, pp. 993–1001, 1990.